8. Performance & Sustainability Analysis

Objective: Assess runtime performance and sustainability across algorithms, data, API, memory, CPU, concurrency, caching, resources, network, build, logging, and energy efficiency; recommend efficiency and cost/carbon improvements.

Date: 2026-07-24 | **Scope:** `shende-shweta/FSDKC` (`main`) — PHP 8.3 / Laravel 12 API, React 19.2 + TypeScript + Vite SPA, Node/Express `dev-api` , MariaDB 11 + MongoDB 7, Docker Compose (nginx/php-fpm) + GitHub Actions CI (EC2/CodeDeploy pattern)

Executive Summary

EXECUTIVE SUMMARY

Klearcom's dual runtime (Laravel + Node `dev-api`) is a small voice-observability platform with clear efficiency debt concentrated in data access, long-lived streaming, and always-on Docker resources. The highest-risk patterns are an N+1 query loop in `LegacyReportController` , unbounded MariaDB list loads, SSE endpoints that re-read full Mongo event histories every 500 ms while holding request workers with `usleep` , and a five-service Compose stack with no CPU/memory limits that ships the frontend via `npm run dev` . Frontend polling (`refetchInterval` , `LegacyMonitorPoller` every 3 s, `MongoStatus` every 15 s) amplifies traffic while nginx lacks gzip. CI installs Composer/npm and pecl MongoDB with no dependency caching. Overall rating is **High Risk**, driven by database, API latency, memory retention, concurrency, network chatter, resource waste, and sustainability posture.

39	3	6	6
FILES / FUNCTIONS SCANNED	HIGH-COMPLEXITY FUNCTIONS	N+1 / SLOW-QUERY SITES	BLOCKING I/O SITES

OVERALL CODEBASE RATING — PERFORMANCE & SUSTAINABILITY

High Risk

Driven by P2 Database, P3 API, P4 Memory, P6 Concurrency, P8 Resource Utilization, P9 Network, and P12 Sustainability.

8.1 Benchmark Ratings Summary

Application sources scanned: **39 files** (backend 18 · frontend 15 · `dev-api` 6), plus Docker/CI/nginx manifests for P8–P12. Counts are absolute pattern sites from GitHub `main` reads — not invented.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating

P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	3	MODERATE
P2	Database Performance	Slow-query / N+1 sites	0	1–5	>5	6	HIGH RISK
P3	API Performance	Response-latency hotspots	0	1–5	>5	6	HIGH RISK
P4	Memory Efficiency	High-memory sites	0	1–3	>3	4	HIGH RISK
P5	CPU Efficiency	CPU-intensive operations	0	1–5	>5	3	MODERATE
P6	Concurrency	Blocking / sequential sites	0	1–5	>5	6	HIGH RISK
P7	Caching	Missed caching opportunities	0	1–5	>5	5	MODERATE
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	4	HIGH RISK
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	6	HIGH RISK
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	partial	MODERATE
P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	1	MODERATE
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	wasteful	HIGH RISK
P13	Missing client timeouts (additional)	Fetch/HTTP calls without timeout	0	1–2	>2	1	MODERATE
P14	Uncleared polling intervals (additional)	Intervals without unmount cleanup	0	1	≥2	1	MODERATE

8.2 Hotspot Analysis

P1. Algorithm Efficiency MEDIUM

Benchmark: `High-complexity algorithm sites = 3` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

Three IVR `buildTree` implementations repeatedly scan the full node collection at every recursive level (O(n²) worst case as trees deepen). Sites: `DiscoveryController::buildTree`, `LegacyReportController::buildTree`, and `dev-api/src/store.js` `buildTree`.

Example — Laravel Discovery tree rebuild filters the whole collection per parent:

```
    private function buildTree($nodes, ?int $parentId = null): array
    {
        return $nodes
            ->where('parent_id', $parentId)
            ->map(fn (DiscoveryNode $node) => [
                'id' => $node->id,
                'prompt_text' => $node->prompt_text,
                'dtmf_option' => $node->dtmf_option,
                'node_type' => $node->node_type,
                'depth' => $node->depth,
                'children' => $this->buildTree($nodes, $node->id),
            ])
            ->values()
            ->all();
    }
```

Example — identical O(n²) pattern in the Node store:

```
export function buildTree(nodes, parentId = null) {
  return nodes
    .filter((n) => n.parent_id === parentId)
    .map((n) => ({
      id: n.id,
      prompt_text: n.prompt_text,
      dtmf_option: n.dtmf_option,
      node_type: n.node_type,
      depth: n.depth,
      children: buildTree(nodes, n.id),
    }));
}
```

Why it matters here: Discovery jobs map multi-level IVR menus; as `nodes_discovered` grows beyond seed sizes (12+), each `/tree` and legacy IVR report pays a quadratic scan. That latency hits the UI tree panel, which also refetches every 2 s while a test runs.

Recommended approach: (1) Group nodes once by `parent_id` into a hash map, then build children in O(n). (2) Extract a shared tree builder used by Laravel and `dev-api`. (3) Cache the serialized tree per `job_id` until nodes change.

No files matched `**/*.{php,js}` containing `function buildTree|private function buildTree`.

P2. Database Performance CRITICAL

Benchmark: `Slow-query / N+1 sites = 6` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

Six sites: (1) N+1 in `LegacyReportController::carrierSummary`, (2–3) unbounded `DiscoveryJob::get()` / `ConnectMonitor::get()`, (4) `DashboardController` six separate

COUNT/AVG queries, (5) SSE/ `getTestEvents` re-reading all Mongo events without cursor, (6) no secondary indexes on `status` / `country_code` / `checked_at` in `init.sql` (filter/sort hot paths).

Example — classic N+1: one query per monitor inside the loop:

```
        $monitors = ConnectMonitor::query()
            ->when(isset($country_code), fn ($q) => $q->where('country_code',
$country_code))
            ->when(isset($carrier), fn ($q) => $q->where('carrier', $carrier))
            ->orderByDesc('reachability_pct')
            ->get();

        $rows = [];
        $mapper = new LegacyDataMapper();

        foreach ($monitors as $monitor) {
            $recent = ConnectCheckResult::where('connect_monitor_id', $monitor-
>id)
                ->orderByDesc('checked_at')
                ->limit(20)
                ->get();
```

Example — unbounded list load (no pagination):

```
    public function index(): JsonResponse
    {
        $jobs = DiscoveryJob::orderByDesc('created_at')->get();

        return response()->json(['data' => $jobs]);
    }
```

Schema excerpt — FKs present, but no indexes for status/country filters used by dashboard and reports:

```
CREATE TABLE IF NOT EXISTS connect_monitors (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    toll_free_number VARCHAR(50) NOT NULL,
    country_code VARCHAR(5) NOT NULL,
    carrier VARCHAR(100) NULL,
    status ENUM('active', 'paused', 'alert') DEFAULT 'active',
    reachability_pct DECIMAL(5,2) DEFAULT 100.00,
    last_checked_at TIMESTAMP NULL,
```

Why it matters here: Carrier summary and dashboard KPIs are operational hot paths. With hundreds of TFN monitors and check history growth, N+1 plus missing filter indexes dominate MariaDB round-trips; unbounded `get()` inflates payload and memory on every list view.

Recommended approach: (1) Eager-load or batch recent checks with a window/ `whereIn` . (2) Paginate `index` endpoints. (3) Collapse dashboard KPIs into one aggregated query (or cached snapshot). (4) Add indexes on `(status)` , `(country_code)` , `(connect_monitor_id, checked_at)` . (5) Use Mongo change streams or tailable cursors for SSE instead of full finds.

No files matched `backend/app/**/*.php` *containing* `foreach\s*\(\s*\$monitors|::orderByDesc\([^)]+\)->get\(\)|ConnectCheckResult::where` .

P3. API Performance CRITICAL

Benchmark: `Response-latency hotspots = 6` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

Sites: blocking SSE poll in `StreamController` , `usleep` simulated tests in `RealTimeTestService` , `sleep` loops in `dev-api/src/realtime.js` , unbounded list payloads, dashboard multi-query latency, and frontend multi-endpoint refetch during live tests (jobs + tree + transcripts).

Example — PHP-FPM worker held open, re-querying Mongo every 500 ms:

```
        return response()->stream(function () use ($sessionId): void {
            $sent = 0;

            foreach ($this->mongo->getTestEvents($sessionId) as $event) {
                echo 'data: '.json_encode($event)."\n\n";
                ob_flush();
                flush();
                $sent++;
                if (($event['event']['type'] ?? '') === 'complete') {
                    return;
                }
            }

            $attempts = 0;
            while ($attempts < 60) {
                $events = $this->mongo->getTestEvents($sessionId);
                foreach (array_slice($events, $sent) as $event) {
                    echo 'data: '.json_encode($event)."\n\n";
```

Example — sequential artificial delay on the worker path:

```
        $parentId = null;
        foreach ($steps as $step) {
            usleep(600_000);
            $this->mongo->storeTestEvent($sessionId, 'discovery', $jobId,
['type' => 'step', ...$step]);
```

Why it matters here: Each live Discovery/Connect test opens an SSE connection and runs multi-second sleep loops. Under concurrent testers, PHP-FPM/ `afterResponse` closures and Node event loops serialize work; list APIs without pagination grow response times as job/monitor tables grow.

Recommended approach: (1) Move test simulation to a queue worker (Redis/SQS). (2) Push events via Redis pub/sub or Mongo change streams instead of polling. (3) Paginate list APIs and trim show payloads. (4) Rely on SSE alone for live updates—disable parallel REST refetch during runs.

No files matched `**/*.{php,js,tsx,ts}` *containing* `usleep|while\s*\(\s*\$attempts|getTestEvents|refetchInterval` .

P4. Memory Efficiency HIGH

Benchmark: `High-memory sites = 4` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

Sites: unbounded in-memory `store` arrays in `dev-api`, `getTestEvents` loading entire session history with no limit, unbounded Eloquent `get()` collections, and `LegacyMonitorPoller` interval leak retaining component/closure memory after unmount.

Example — Mongo events loaded fully into PHP arrays on every poll:

```
    public function getTestEvents(string $sessionId): array
    {
        if ($this->testEvents === null) {
            return [];
        }

        $cursor = $this->testEvents->find(
            ['session_id' => $sessionId],
            ['sort' => ['created_at' => 1]]
        );

        return array_map([$this, 'serializeDocument'],
iterator_to_array($cursor));
```

Example — process-lifetime arrays that only grow:

```
/** In-memory relational store (MariaDB equivalent for local dev) */

export const store = {
  discoveryJobs: [
```

Why it matters here: Long-running `dev-api` processes and SSE sessions accumulate nodes, checks, and serialized events. `iterator_to_array` materializes the full cursor each poll, multiplying heap use under concurrent streams.

Recommended approach: (1) Cap `getTestEvents` with `skip` / `limit` or resume tokens. (2) Bound in-memory store growth or use MariaDB for persistence. (3) Paginate Eloquent lists. (4) Clear intervals on unmount in `LegacyMonitorPoller`.

No files matched `**/*.{php,js,jsx}` *containing* `getTestEvents|iterator_to_array|export const store|setInterval` .

P5. CPU Efficiency MEDIUM

Benchmark: `CPU-intensive operations = 3` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

Sites: repeated `json_encode` of full event documents on every SSE tick, O(n²) `buildTree` collection filters on hot `/tree` paths, and `countDocuments` on three collections inside every Mongo health/status check.

Example — health path counts entire collections:

```
export async function healthCheck() {
  try {
    const result = await getDb().command({ ping: 1 });
    const transcripts = await
getDb().collection('transcripts').countDocuments();
    const events = await getDb().collection('test_events').countDocuments();
    const diagnostics = await
getDb().collection('call_diagnostics').countDocuments();
```

Mirrored in Laravel `MongoService::health()` with three `countDocuments()` calls, polled by the sidebar every 15 s.

Why it matters here: Status widgets and SSE encode/serialize continuously. Collection counts become CPU+IO heavy as transcript/event volumes grow, competing with write-heavy live tests.

Recommended approach: (1) Ping-only health; expose counts on a separate, cached admin endpoint. (2) Encode only new events for SSE. (3) Fix tree builder to O(n) (see P1).

No files matched `**/*.{php,js}` *containing* `countDocuments|json_encode\($event\)|buildTree` .

P6. Concurrency CRITICAL

Benchmark: `Blocking / sequential sites = 6` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

Sites: `StreamController` `usleep` loop occupying PHP workers, `RealTimeTestService` `usleep` after response, Node `setInterval` SSE poll, sequential `await storeTestEvent` in realtime loops, no dedicated queue worker, and `LegacyMonitorPoller` leaking concurrent timers.

Example — Node SSE poll every 500 ms re-fetching events:

```
async function streamSession(res, req, sessionId) {
  setupSse(res);
  let sent = 0;

  const poll = setInterval(async () => {
    try {
      const events = await getTestEvents(sessionId);
      for (const evt of events.slice(sent)) {
        sendSse(res, serializeDoc(evt));
        sent++;
        if (evt.event?.type === 'complete') {
          clearInterval(poll);
          setTimeout(() => res.end(), 400);
        }
```

```
      }
    } catch {
      clearInterval(poll);
      res.end();
    }
  }, 500);
```

Example — tests dispatched but still sleep-bound on the app process:

```
        dispatch(function () use ($id, $sessionId): void {
            app(RealTimeTestService::class)->runDiscoveryTest($id, $sessionId);
        })->afterResponse();
```

Why it matters here: Concurrent Discovery and Connect tests multiply blocking SSE workers and sequential Mongo writes. Without a queue/worker pool, throughput collapses as sessions increase; leaked intervals keep issuing work after navigation away.

Recommended approach: (1) Use Laravel queues + Horizon/Redis for `runDiscoveryTest` / `runConnectTest`. (2) Replace poll loops with pub/sub push. (3) Parallelize independent Mongo writes where safe. (4) Size PHP-FPM/ `pm.max_children` for remaining long-lived streams. (5) Fix poller cleanup (P14).

No files matched `**/*.{php,js,jsx,tsx}` containing `usleep|setInterval|afterResponse|runDiscoveryTest|runConnectTest`.

P7. Caching MEDIUM

Benchmark: `Missed caching opportunities = 5` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

No Redis/ `Cache::` usage observed. Missed sites: dashboard KPI recomputation every request, Mongo health counts every status poll, IVR tree rebuilt every GET, no HTTP `Cache-Control` /ETag on read APIs, nginx without static/API cache headers.

Example — six DB aggregations with no cache layer:

```
    public function kpis(): JsonResponse
    {
        $discoveryTotal = DiscoveryJob::count();
        $discoveryCompleted = DiscoveryJob::where('status', 'completed')-
>count();
        $connectMonitors = ConnectMonitor::count();
        $avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
        $alerts = ConnectMonitor::where('status', 'alert')->count();
        // ...
                'active_discovery_jobs' => DiscoveryJob::where('status',
'running')->count(),
                'active_connect_monitors' => ConnectMonitor::where('status',
'active')->count(),
```

Frontend React Query (`staleTime: 30_000` in `main.tsx`) helps the SPA, but legacy widgets and status endpoints bypass meaningful server-side caching.

Why it matters here: Dashboard and Mongo status are read-heavy and mostly eventually consistent. Recomputing counts and trees on every hit wastes MariaDB/Mongo capacity that live tests need.

Recommended approach: (1) Cache KPI JSON 30–60 s in Redis/file cache with invalidation on job/monitor writes. (2) Cache tree by `job_id` + `nodes_discovered` version. (3) Cache health ping result briefly. (4) Add short `Cache-Control` on safe GETs.

No files matched `backend/app/**/*.php` *containing* `function kpis|function health|function tree\(|buildTree` .

P8. Resource Utilization HIGH

Benchmark: `Over-provisioned / idle resource configs = 4` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

Configs: (1) five always-on Compose services with no `deploy.resources` / memory-CPU limits, (2) frontend container runs Vite **dev** server not a production build, (3) `APP_DEBUG: "true"` in Compose, (4) MariaDB/Mongo ports published to host continuously (3306/27017).

```
services:
  nginx:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"
  app:
    environment:
      APP_ENV: local
      APP_DEBUG: "true"
  mariadb:
    ports:
      - "3306:3306"
  mongodb:
    ports:
      - "27017:27017"
```

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev"]
```

Why it matters here: README targets EC2/CodeDeploy-style deploy. Running debug PHP-FPM, unbound databases, and a Vite HMR server burns idle CPU/RAM and energy even with no users — opposite of right-sizing/autoscaling.

Recommended approach: (1) Add memory/CPU limits and health-based restart policies. (2) Multi-stage frontend image serving static assets via nginx. (3) `APP_DEBUG=false` outside local profiles. (4) Stop publishing DB ports in non-dev profiles; consider compose profiles for optional services.

No files matched `docker-compose.yml` *containing* `APP_DEBUG|npm run dev|ports:` .

P9. Network Efficiency HIGH

Benchmark: `Excessive-traffic sites = 6` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

Sites: Discovery/Connect `refetchInterval` 1.5–2 s on three endpoints while SSE already streams, `LegacyMonitorPoller` 3 s checks poll, `MongoStatus` 15 s status (with full counts), `LegacyDashboardWidget` 10 s KPI poll, nginx without gzip/brotli, duplicate transcript fetches (show endpoint + separate `/mongodb/transcripts`).

Example — chatty client while SSE is active:

```
  const jobsQuery = useQuery({
    queryKey: ['discovery', 'jobs'],
    queryFn: () => api.get<{ data: DiscoveryJob[] }>('/discovery/jobs'),
    refetchInterval: isRunning ? 2000 : false,
  });

  const treeQuery = useQuery({
    queryKey: ['discovery', 'tree', selectedId],
    queryFn: () => api.get<{ tree: DiscoveryNode[] }>
(`/discovery/jobs/${selectedId}/tree`),
    enabled: selectedId !== null,
    refetchInterval: isRunning ? 2000 : false,
  });

  const transcriptsQuery = useQuery({
    queryKey: ['mongodb', 'transcripts', 'discovery', selectedId],
    queryFn: () => api.get<{ data: Transcript[] }>(`/mongodb/transcripts?
module=discovery&reference_id=${selectedId}`),
    enabled: selectedId !== null,
    refetchInterval: isRunning ? 1500 : false,
  });
```

Example — nginx has no compression directives:

```
server {
    listen 80;
    server_name localhost;
    root /var/www/html/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
```

Why it matters here: During a single live test the browser can exceed one request/second across jobs/tree/transcripts/checks plus SSE, multiplying bandwidth and backend load. Uncompressed JSON SSE frames add unnecessary bytes.

Recommended approach: (1) Drive UI from SSE events only during runs. (2) Remove or gate legacy pollers. (3) Enable `gzip` / `brotli` in nginx. (4) Coalesce status into one lightweight endpoint.

No files matched `**/*.{tsx,jsx,ts,conf}` containing `refetchInterval|setInterval|gzip`.

P10. Build Efficiency MEDIUM

Benchmark: `Build/test pipeline efficiency = partial` → falls in the **Moderate** band (efficient · partial · slow / no caching).

`.github/workflows/ci.yml` runs backend and frontend jobs in parallel (good) but: no `actions/cache` for Composer or npm, `setup-node` omits `cache: npm`, and every backend job runs `sudo pecl install mongodb` from scratch.

```
  backend:
    runs-on: ubuntu-latest
    ...
      - name: Install MongoDB extension
        run: |
          sudo pecl install mongodb
          echo "extension=mongodb.so" | sudo tee -a "$(php -r 'echo
PHP_CONFIG_FILE_SCAN_DIR;')"/mongodb.ini
      - run: composer install --no-interaction --prefer-dist
        working-directory: backend
  frontend:
    ...
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm ci || npm install
        working-directory: frontend
```

Why it matters here: Each push rebuilds PHP extensions and downloads dependencies cold, lengthening feedback loops and burning CI compute/energy on unchanged lockfiles.

Recommended approach: (1) Cache Composer and npm via `actions/cache` / `setup-node` cache. (2) Prefer a prebuilt PHP image with mongodb ext. (3) Skip frontend build when `frontend/**` unchanged (path filters).

No files matched `.github/workflows/*.yml` containing `pecl install|composer install|npm ci`.

P11. Logging Efficiency MEDIUM

Benchmark: `Excessive-logging sites = 1` → falls in the **Moderate** band (Good 0 · Moderate 1–10 · High Risk >10).

No `Log::` calls inside hot loops were observed. The measured site is Compose forcing `APP_DEBUG: "true"`, which enables verbose Laravel error/stack output on the request path for the containerized stack. Boot `console.log` in `dev-api` is low volume and not counted as hot-loop logging.

```
    environment:
      APP_ENV: local
      APP_DEBUG: "true"
```

Evidence: Not a high volume of loop logging; risk is debug-mode verbosity when the Compose file is reused beyond local.

Recommended approach: Keep debug off in shared/staging Compose profiles; use structured logging with sampling for production SSE/test paths.

No files matched `docker-compose.yml` *containing* `APP_DEBUG` .

P12. Sustainability HIGH

Benchmark: `Resource-optimization posture = wasteful` → falls in the **High Risk** band (optimized · partial · wasteful).

Always-on five-service Compose, Vite dev server in Docker, N+1/polling inefficiencies, CI cold installs, and no carbon-aware or spot/serverless scheduling evidence. Positive notes: nginx Alpine image, Mongo indexes created in `mongo.js` for common filters, React Query staleTime present.

Why it matters here: Energy and cost scale with idle containers, chatty polls, and blocking workers—not with useful IVR/TFN work. Without right-sizing and efficient request paths, carbon and cloud spend grow with uptime rather than traffic.

Recommended approach: (1) Prod-oriented Compose profile (static FE, debug off, resource limits). (2) Fix P2/P3/P6/P9 hotspots to cut wasted compute. (3) Cache CI deps. (4) Plan off-peak batch monitoring and autoscaled workers when moving to AWS.

No files matched `docker-compose.yml` *containing* `APP_DEBUG|npm run dev|services:` .

P13. Missing client timeouts (additional) MEDIUM

Benchmark: `Fetch/HTTP calls without timeout = 1` → falls in the **Moderate** band (Good 0 · Moderate 1–2 · High Risk >2). KPI justification: unbounded client waits stall UI threads and keep sockets open under slow APIs.

```
async function request<T>(path: string, options?: RequestInit): Promise<T> {
  const res = await fetch(`${API_BASE}${path}`, {
    headers: { 'Content-Type': 'application/json', Accept: 'application/json'
},
    ...options,
  });

  if (!res.ok) {
    const body = await res.text();
    throw new Error(body || `HTTP ${res.status}`);
  }
```

Why it matters here: List and status calls can hang indefinitely if nginx/php-fpm stalls (e.g., saturated by SSE workers), leaving the SPA in perpetual loading states and holding browser connections.

Recommended approach: Wrap `fetch` with `AbortSignal.timeout(...)` (or AbortController + timer); surface timeout errors to the UI.

No files matched `frontend/src/api/**/*.ts` *containing* `async function request|fetch\(\`\${API_BASE}` .

P14. Uncleared polling intervals (additional) MEDIUM

Benchmark: `Intervals without unmount cleanup = 1` → falls in the **Moderate** band (Good 0 · Moderate 1 · High Risk ≥2). KPI justification: leaked timers keep issuing network/CPU work after the component is gone.

```
  componentDidMount() {
    this.intervalId = setInterval(() => {
      api.get<{ computed?: { reachability_pct: number } }>
(`/connect/monitors/${this.props.monitorId}/checks`)
        .then((res) => {
          const pct = res.computed?.reachability_pct ?? null;
          this.setState({ reachability: pct, error: null });
          if (pct != null) this.props.onUpdate?.(pct);
        })
        .catch((err: Error) => this.setState({ error: err.message }));
    }, 3000);
    // Intentionally no componentWillUnmount — EventSource/interval leak for
audit finding
  }
```

Why it matters here: Every mounted poller continues hitting `/checks` every 3 s forever, compounding P9 network load and retaining component state (P4).

Recommended approach: Implement `componentWillUnmount` / useEffect cleanup to `clearInterval`; prefer React Query with `refetchInterval` that auto-cancels.

No files matched `frontend/src/components/**/*.{jsx,tsx}` containing `setInterval|componentWillUnmount|LegacyMonitorPoller`.

8.3 Runtime Architecture

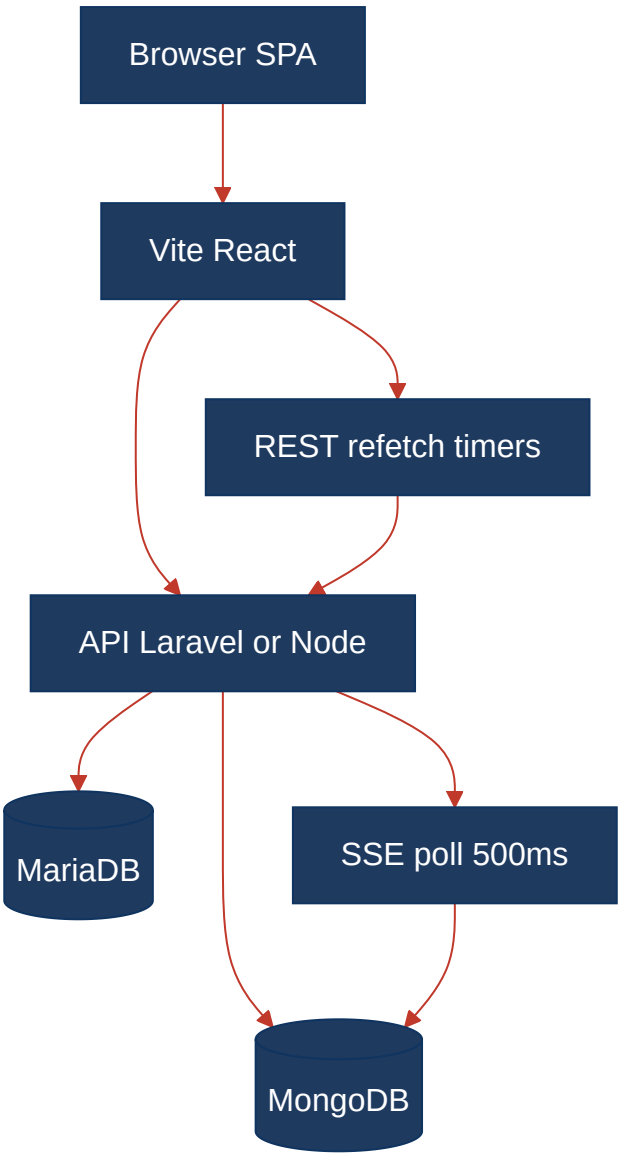
Primary local path (`npm run dev`): Browser (React 19 + Vite :5173) → HTTP/JSON + EventSource → Node Express `dev-api` (:8080) → in-memory relational `store` + MongoDB (Atlas URI or in-memory server). Live Discovery/Connect tests write `test_events` / `transcripts` / `call_diagnostics`, while SSE handlers poll Mongo every 500 ms. Frontend additionally refetches REST resources on short intervals during runs.

Docker/full-stack path: Browser → Vite frontend container → nginx (:8080) → PHP-FPM Laravel 12 (`docker/php`) → MariaDB 11 (relational jobs/monitors/checks) + MongoDB 7 (transcripts/events). `RealTimeTestService` runs after the HTTP response via `dispatch(...)->afterResponse()`, still sleeping and writing on the app process. Hotspots sit on: list/KPI controllers (P2/P7), stream endpoints (P3/P4/P6), legacy report N+1 (P2), and Compose resource posture (P8/P12).

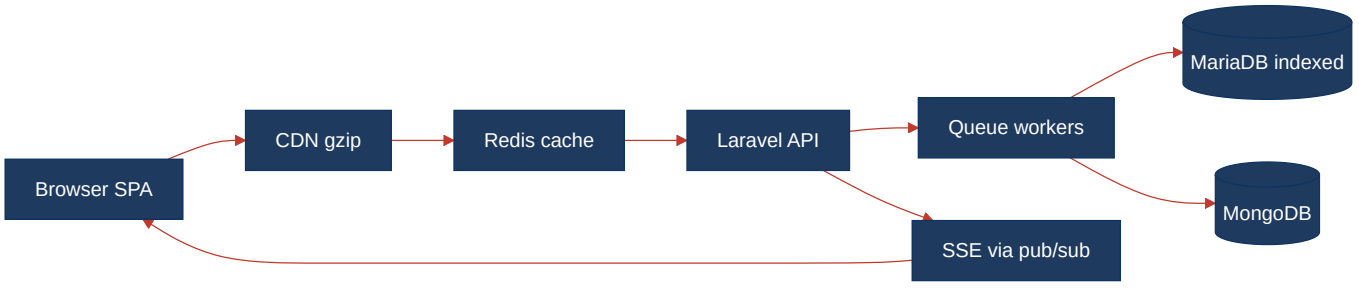
Sustainability/cost posture: Always-on multi-container stack with debug on and no autoscaling or resource limits; no carbon-aware scheduling in repo. Energy-relevant choices today are Alpine nginx (good) versus Vite-dev-in-Docker and chatty polling (wasteful).

8.4 Diagrams

Current runtime flow

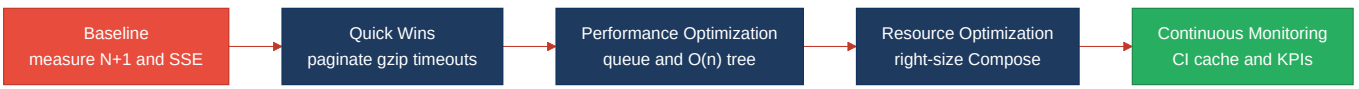


Optimized runtime target



Sustainability optimization roadmap

Derived from the Actions Required priorities below (Critical first).



8.5 Actions Required

Hotspot	Action	Rating	Priority
P2 Database Performance	Eliminate N+1 in `LegacyReportController`; paginate list endpoints; add indexes on status/country/checked_at; cursor-limit Mongo event reads	HIGH RISK	CRITICAL
P3 API Performance	Move simulated tests to queue workers; replace SSE Mongo re-poll with pub/sub; stop parallel REST refetch during live runs	HIGH RISK	CRITICAL
P6 Concurrency	Stop holding PHP-FPM/Node with `usleep` /interval polls; introduce worker pool and push-based streams	HIGH RISK	CRITICAL
P4 Memory Efficiency	Cap `getTestEvents` and in-memory store growth; paginate Eloquent collections	HIGH RISK	HIGH
P8 Resource Utilization	Add Compose resource limits; ship static frontend image; disable APP_DEBUG outside local; unpublish DB ports in non-dev	HIGH RISK	HIGH
P9 Network Efficiency	Prefer SSE-only live updates; remove legacy 3 s/10 s pollers; enable nginx gzip/brotli	HIGH RISK	HIGH
P12 Sustainability	Right-size always-on stack and cut wasteful poll/N+1 paths; plan autoscaled workers for AWS	HIGH RISK	HIGH
P1 Algorithm Efficiency	Replace recursive full-scan `buildTree` with parent_id hash grouping (O(n)) shared across Laravel and `dev-api`	MODERATE	MEDIUM
P5 CPU Efficiency	Ping-only health checks; avoid full collection counts on the 15 s status path	MODERATE	MEDIUM
P7 Caching	Cache dashboard KPIs and IVR trees; short-circuit repeated health work	MODERATE	MEDIUM
P10 Build Efficiency	Cache Composer/npm and avoid pecl rebuild every CI run	MODERATE	MEDIUM
P11 Logging Efficiency	Default `APP_DEBUG=false` outside local Compose profile	MODERATE	MEDIUM
P13 Missing client timeouts	Add `AbortSignal.timeout` to `frontend/src/api/client.ts`	MODERATE	MEDIUM
P14 Uncleared polling intervals	Clear `LegacyMonitorPoller` interval on unmount	MODERATE	MEDIUM

8.6 Expected Outcomes

- Batched/indexed MariaDB access and paginated lists remove N+1 and unbounded payload latency as monitors and jobs scale.
- Queue-backed tests plus pub/sub SSE free PHP-FPM/Node workers, raising concurrent Discovery/Connect throughput.
- O(n) tree builds, capped Mongo reads, and server-side KPI/tree caches cut CPU, memory, and repeated query cost.
- gzip, SSE-only live updates, and removal of leaked/legacy pollers reduce chatty traffic and bandwidth.
- Right-sized Compose (static FE, debug off, limits) plus CI dependency caching lower cloud cost, idle energy use, and carbon footprint.